Care Access Navigator

CloudSec1 Rubric — Criterion 3: Cloud Infrastructure Security & Container Protection

Evidence Package — VM Hardening, Container Security (Docker + Trivy) & Serverless Function Isolation

Prepared by:	Akhila Chennamaneni
Capstone:	Care Access Navigator (HealthLine Regional Medical Center)
Rubric criterion:	Criterion 3 — Cloud Infrastructure Security & Container Protection (CloudSec1)
Environment:	Microsoft Azure (Azure subscription 1, resource group rg-cloudsec1-can); Docker 29.7 + Trivy 0.75 on macOS
Evidence dates:	October 6–8, 2026

Objective

This criterion requires practical skills securing three kinds of cloud compute: one virtual machine (automated patching, host firewall, secure SSH), one containerised workload (secure Dockerfile, Docker security settings, and container scanning with documented results) and one serverless function (proper IAM roles and network isolation). Each control below was implemented on real Azure resources or on the real CAN EHR simulator container, then **tested** — including negative tests showing that what should be blocked is blocked.

Scope of this evidence package

- **Part A — VM hardening:** Ubuntu 24.04 VM with Trusted Launch, Azure-managed patching + unattended-upgrades, NSG + ufw host firewall, key-only SSH, fail2ban
- **Part B — Container security:** hardened multi-stage Dockerfile, hardened Docker runtime settings, Trivy image/config/secret scans before and after (172 → 0 vulnerabilities)
- **Part C — Serverless:** Azure Function (Flex Consumption) with a user-assigned managed identity, single least-privilege role, no keys, VNet integration, storage firewall and IP access restrictions
- Genuine issues encountered and how they were resolved (documented, not hidden)
- Compliance mapping to HIPAA Security Rule and NIST CSF 2.0

Cost constraint: All resources were built under a strict cost constraint (pay-as-you-go on developer credit). The VM was created temporarily for this evidence and deallocated/deleted afterwards; the container work ran locally at no cost; the function runs on the Flex Consumption free grant.

Results at a Glance

Requirement	Implementation	Verified result
VM: automated patching	Azure AutomaticByPlatform patching + unattended-upgrades (security origins, daily, auto-reboot 03:00)	**PASS** — security kernel update auto-installed; Azure assessment: 0 security/critical outstanding
VM: host firewall	NSG vm-sg (SSH from admin IP only, deny-all) + ufw default-deny, rate-limited SSH	**PASS** — IP flow verify: admin IP allowed, other IP denied, port 80 denied
VM: secure SSH	Key-only (ed25519), no root, AllowUsers canadmin, MaxAuthTries 3, modern ciphers, no forwarding, fail2ban	**PASS** — password, root and unknown-user logins all rejected
Container: secure Dockerfile	Multi-stage, digest-pinned Chainguard base, non-root UID 65532, no shell/pip, read-only files, gunicorn	**PASS** — Trivy: 172 → **0** vulnerabilities; 0 misconfigurations
Container: Docker settings	read_only, cap_drop ALL, no-new-privileges, pids/mem/cpu limits, localhost-only port	**PASS** — writes blocked, CapEff=0, fork bomb stopped at 96
Container: scanning	Trivy vuln + secret + Dockerfile config scans, CycloneDX SBOM	**PASS** — reports saved in can-ehr-container/reports/
Serverless: IAM	User-assigned managed identity, one role (Storage Blob Data Owner) on one storage account; shared keys disabled	**PASS** — function reads storage with no secrets; key auth refused
Serverless: network isolation	VNet integration (snet-func), storage firewall (subnet + admin IP), function IP restrictions, HTTPS/TLS 1.2	**PASS** — 401 without key, 403 from non-allowed IP even with valid key

Architecture

Component	Value
Resource group	rg-cloudsec1-can
VM	can-vm-hardened — Ubuntu 24.04 LTS, Standard_F1als_v7, North Central US, Trusted Launch (Secure Boot + vTPM)
VM network	can-vnet-vmlab / snet-vm (10.30.1.0/24), NSG vm-sg
Container	can-ehr-simulator:2.0 (Chainguard/Wolfi Python 3.14, gunicorn), run via docker-compose.secure.yml
Function App	func-can-ehr-9a8c96 — Python 3.12, Flex Consumption, East US
Function identity	id-can-func (user-assigned managed identity)
Function network	can-vnet-cloudsec1 / snet-func (10.20.5.0/27, delegated Microsoft.App/environments, NSG func-sg, Microsoft.Storage service endpoint)
Function storage	stcanfunc9a8c964b — shared-key access disabled, TLS 1.2, no anonymous access, default-deny firewall

Part A — Virtual Machine Hardening

A dedicated Ubuntu 24.04 LTS VM was deployed with Azure platform protections enabled at creation, then hardened at the operating-system level with a repeatable script (`can-vm-hardening/harden.sh`) run over SSH.

STEP A1 Deploy the VM with platform security enabled

The VM was created with SSH-key authentication only (password authentication disabled at the platform level), Trusted Launch (Secure Boot and a virtual TPM to protect the boot chain), and **AutomaticByPlatform** patch orchestration so Azure applies critical and security patches automatically using safe deployment practices.

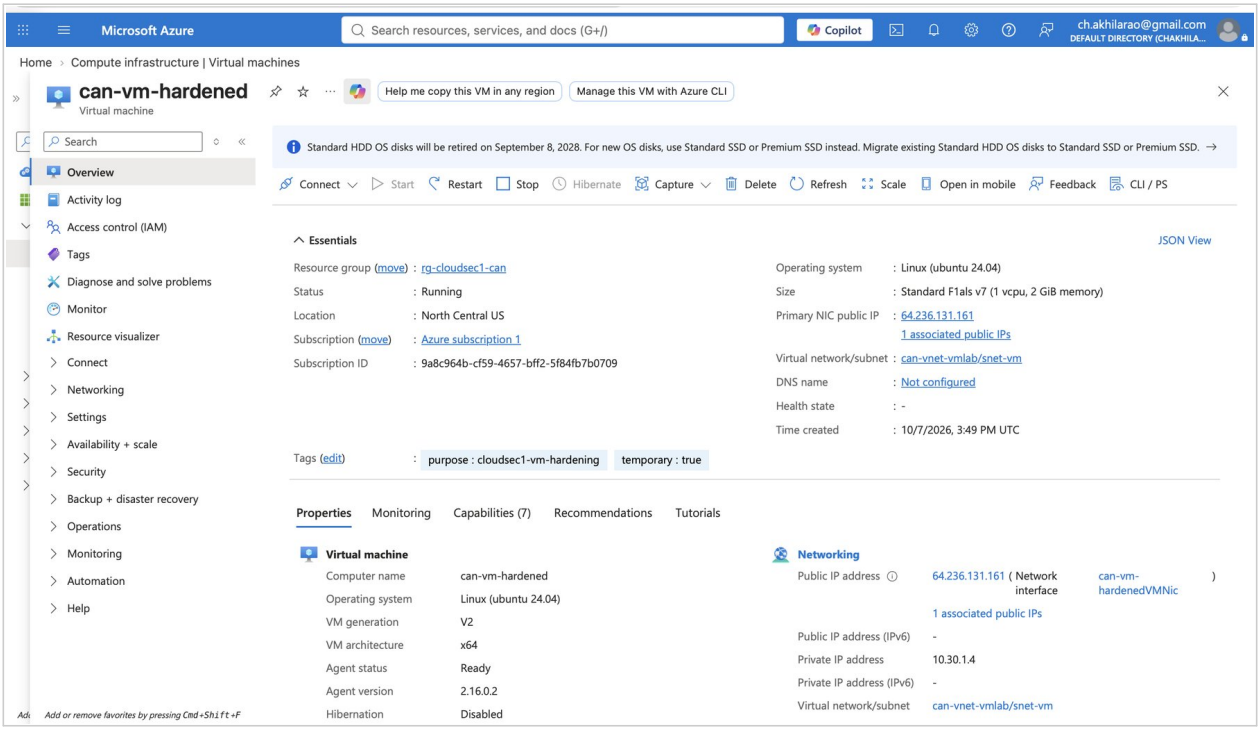


Figure 1. VM overview — can-vm-hardened running Ubuntu 24.04 (Standard F1als v7, North Central US), tagged as temporary for this evidence.

STEP A2 Automated patching — Azure platform + unattended-upgrades

Two layers of automated patching were configured. Azure manages platform patching (*Azure Managed — Safe Deployment*), and inside the OS **unattended-upgrades** installs security updates daily from the security origins only, removes unused dependencies, and reboots at 03:00 when a kernel update requires it. To prove it works end-to-end, unattended-upgrades was run once and installed a pending security kernel update; after reboot the kernel moved from 6.17.0-1022 to 7.0.0-1014 and Azure's patch assessment reported **0 security/critical updates** outstanding.

/etc/apt/apt.conf.d/20auto-upgrades + 52unattended-upgrades-can

```
APT::Periodic::Update-Package-Lists "1";
APT::Periodic::Download-Upgradeable-Packages "1";
APT::Periodic::Unattended-Upgrade "1";
APT::Periodic::AutocleanInterval "7";

Unattended-Upgrade::Allowed-Origins {
    "${distro_id}:${distro_codename}-security";
    "${distro_id}ESMApps:${distro_codename}-apps-security";
    "${distro_id}ESM:${distro_codename}-infra-security";
};
Unattended-Upgrade::Automatic-Reboot "true";
Unattended-Upgrade::Automatic-Reboot-Time "03:00";
```

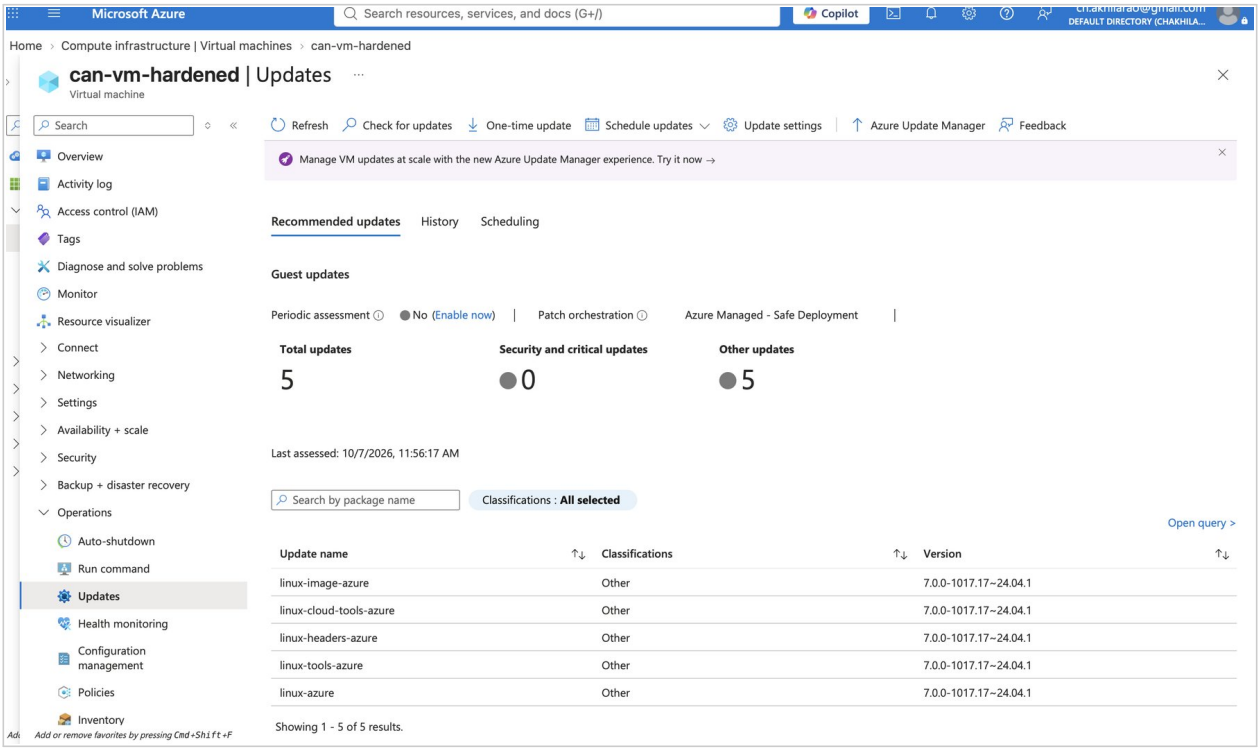


Figure 2. Azure Updates blade — patch orchestration 'Azure Managed — Safe Deployment'; 0 security and critical updates outstanding. The 5 'Other' updates are non-security kernel packages from noble-updates, intentionally excluded by the security-only policy.

STEP A3 Host firewall — Azure NSG and ufw (defence in depth)

Inbound access is filtered twice. At the Azure network layer, NSG **vm-sg** allows TCP 22 only from the administrator's public IP (24.189.77.111/32) and explicitly denies everything else. On the host, **ufw** denies all inbound traffic by default and allows SSH only from the same IP, with rate limiting (LIMIT) to slow brute-force attempts; logging is on.

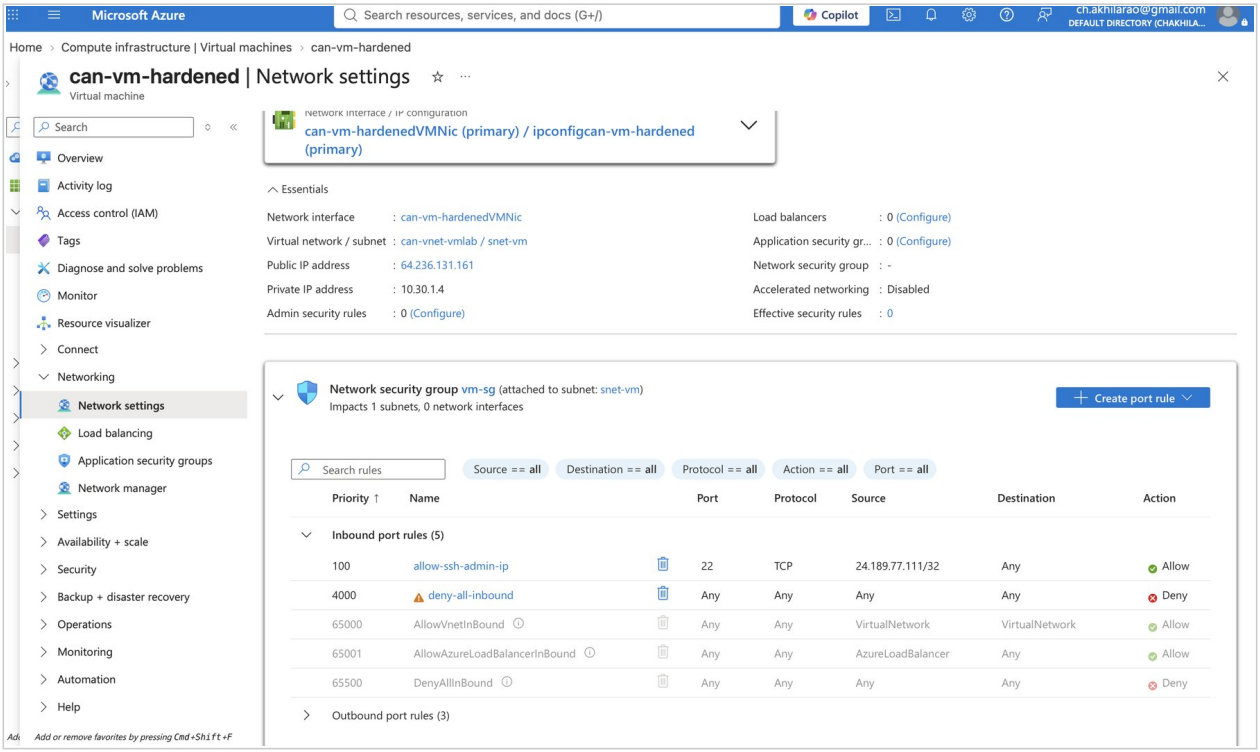


Figure 3. NSG vm-sg — allow-ssh-admin-ip (priority 100, 22/TCP from 24.189.77.111/32) and deny-all-inbound (priority 4000).

The NSG was tested with Azure Network Watcher **IP flow verify**:

Network Watcher IP flow verify

```
SSH from 24.189.77.111 -> Allow securityRules/allow-ssh-admin-ip
SSH from 203.0.113.50 -> Deny securityRules/deny-all-inbound
HTTP 80 from 24.189.77.111 -> Deny securityRules/deny-all-inbound
```

STEP A4 Secure SSH access

SSH hardening is applied in `/etc/ssh/sshd_config.d/00-can-hardening.conf`, which loads before the cloud image defaults so these values take precedence. Root login, passwords and keyboard-interactive auth are disabled; only the **canadmin** account may log in, with public keys only; attempts are limited; forwarding is disabled; only modern key-exchange, cipher and MAC algorithms are offered; and a legal warning banner is shown. **fail2ban** bans an IP for 1 hour after 3 failed attempts in 10 minutes.

SSH hardening (key settings)

```
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
AuthenticationMethods publickey
AllowUsers canadmin
MaxAuthTries 3
LoginGraceTime 30
X11Forwarding no / AllowAgentForwarding no / AllowTcpForwarding no
KexAlgorithms sntrup761x25519-sha512@openssh.com,curve25519-sha256
Ciphers chacha20-poly1305@openssh.com,aes256-gcm@openssh.com,aes128-gcm@openssh.com
MACs hmac-sha2-512-etm@openssh.com,hmac-sha2-256-etm@openssh.com
Banner /etc/issue.net
```

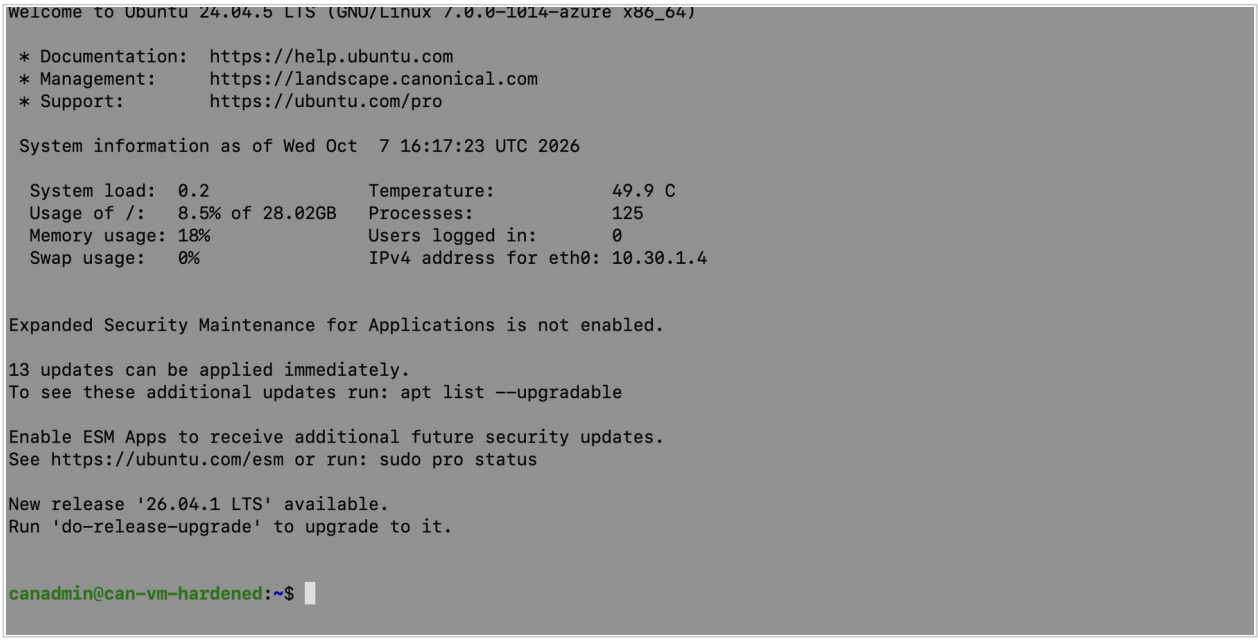


Figure 4. Key-based SSH login as canadmin to can-vm-hardened.

```
Expanded Security Maintenance for Applications is not enabled.

13 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

New release '26.04.1 LTS' available.
Run 'do-release-upgrade' to upgrade to it.


canadmin@can-vm-hardened:~$ sudo ufw status verbose
Status: active
Logging: on (medium)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     LIMIT IN    24.189.77.111              # SSH from admin IP (rate-limited)

canadmin@can-vm-hardened:~$ sudo fail2ban-client status sshd
sudo sshd -T | grep -E "permitroot|passwordauth|allowusers|maxauthtries"
Status for the jail: sshd
|- Filter
|  |- Currently failed: 0
|  |- Total failed:     2
|  `- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
`- Actions
   |- Currently banned: 0
   |- Total banned:     0
   `- Banned IP list:
maxauthtries 3
permitrootlogin no
passwordauthentication no
allowusers canadmin
canadmin@can-vm-hardened:~$
```

Figure 5. Verification on the VM — ufw active (default deny incoming, SSH LIMIT from admin IP only), fail2ban sshd jail active, and effective sshd settings (maxauthtries 3, permitrootlogin no, passwordauthentication no, allowusers canadmin).

STEP A5 Negative tests — what should be blocked is blocked

SSH negative tests (from the admin IP)

```
== Negative SSH tests (from admin IP 24.189.77.111) ==
-- 1. password login attempt (key auth disabled for this test):
canadmin@64.236.131.161: Permission denied (publickey).
-- 2. root login attempt using the valid key:
root@64.236.131.161: Permission denied (publickey).
-- 3. offered ssh auth methods:
Permission denied (publickey)
-- server-side log + fail2ban counters:
Oct 07 15:54:44 can-vm-hardened sshd[1489]: Connection closed by authenticating user canadmin
    24.189.77.111 port 51488 [preauth]
Oct 07 15:54:57 can-vm-hardened sshd[1499]: User root from 24.189.77.111 not allowed because not
    listed in AllowUsers
Oct 07 15:55:09 can-vm-hardened sshd[1504]: User nobody from 24.189.77.111 not allowed because not
     listed in AllowUsers
|  |- Currently failed: 1
|  |- Total failed: 2
   |- Currently banned: 0
   |- Total banned: 0
```

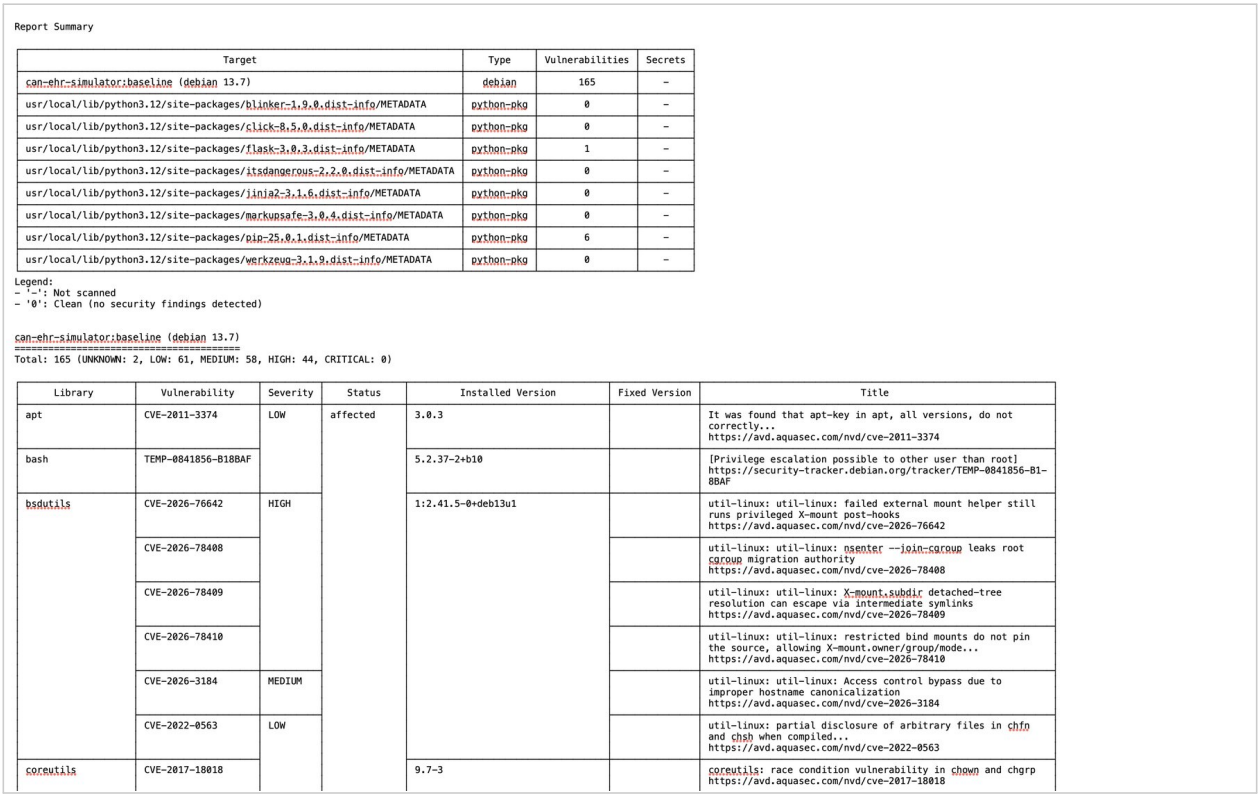
Note: Only two failed attempts were made deliberately: fail2ban bans after three, which would have locked out the administrator's own IP. The server log confirms root and unknown users are refused by AllowUsers, and fail2ban counted both failures.

Part B — Container Security

The workload is CAN's EHR simulator (Flask), previously built from `python:3.12-slim`. It was scanned with **Trivy**, rebuilt with a hardened Dockerfile, run with hardened Docker settings, and scanned again.

STEP B1 Baseline scan — the original image

Trivy found **172 vulnerabilities** in the original image: 165 in Debian OS packages (44 HIGH, 58 MEDIUM, 61 LOW, 2 UNKNOWN) and 7 in Python packages (Flask 3.0.3 and pip 25.0.1). None of the 44 HIGH OS findings had a fix available — they sit in packages the app never uses (util-linux, mount, login), so patching could not remove them; only removing the packages could.



Report Summary

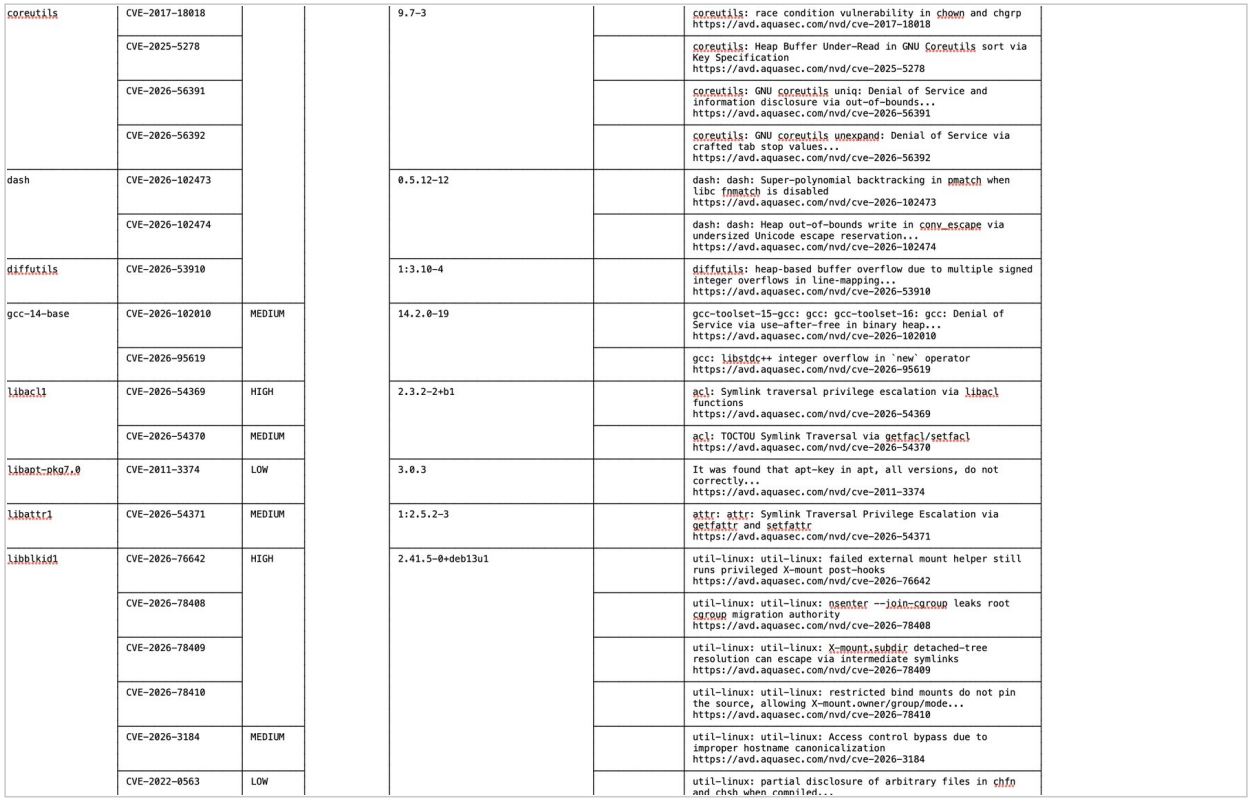
Target	Type	Vulnerabilities	Secrets
can-ehr-simulator:baseline (debian 13.7)	debian	165	-
usr/local/lib/python3.12/site-packages/blinker-1.9.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/click-8.5.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/flask-3.0.3.dist-info/METADATA	python-pkg	1	-
usr/local/lib/python3.12/site-packages/itsdangerous-2.2.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/jinja2-3.1.6.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/markupsafe-3.0.4.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.12/site-packages/pip-25.0.1.dist-info/METADATA	python-pkg	6	-
usr/local/lib/python3.12/site-packages/werkzeug-3.1.9.dist-info/METADATA	python-pkg	0	-

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

can-ehr-simulator:baseline (debian 13.7)
==========
Total: 165 (UNKNOWN: 2, LOW: 61, MEDIUM: 58, HIGH: 44, CRITICAL: 0)

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
apt	CVE-2011-3374	LOW	affected	3.0.3		It was found that apt-key in apt, all versions, do not correctly... https://avd.aquasec.com/nvd/cve-2011-3374
bash	TEMP-0841856-B18BAF			5.2.37-2+b10		[Privilege escalation possible to other user than root] https://security-tracker.debian.org/tracker/TEMP-0841856-B1-8BAF
bsdutils	CVE-2026-76642	HIGH		1:2.41.5-0+deb13u1		util-linux: util-linux: failed external mount helper still runs privileged X-mount post-hooks https://avd.aquasec.com/nvd/cve-2026-76642
	CVE-2026-78408					util-linux: util-linux: nsenter --join-cgroup leaks root cgroup migration authority https://avd.aquasec.com/nvd/cve-2026-78408
	CVE-2026-78409					util-linux: util-linux: X-mount.subdir detached-tree resolution can escape via intermediate symlinks https://avd.aquasec.com/nvd/cve-2026-78409
	CVE-2026-78410					util-linux: util-linux: restricted bind mounts do not pin the source, allowing X-mount.owner/group/mode... https://avd.aquasec.com/nvd/cve-2026-78410
	CVE-2026-3184	MEDIUM				util-linux: util-linux: Access control bypass due to improper hostname canonicalization https://avd.aquasec.com/nvd/cve-2026-3184
	CVE-2022-0563	LOW				util-linux: partial disclosure of arbitrary files in chfn and chsh when compiled... https://avd.aquasec.com/nvd/cve-2022-0563
coreutils	CVE-2017-18018			9.7-3		coreutils: race condition vulnerability in chown and chgrp https://avd.aquasec.com/nvd/cve-2017-18018

Figure 6. Trivy baseline scan — can-ehr-simulator:baseline: 165 OS vulnerabilities, Flask (1) and pip (6) findings.



coreutils	CVE-2017-18018			9.7-3		coreutils: race condition vulnerability in chown and chgrp https://avd.aquasec.com/nvd/cve-2017-18018
	CVE-2025-5278					coreutils: Heap Buffer Under-Read in GNU Coreutils sort via Key Specification https://avd.aquasec.com/nvd/cve-2025-5278
	CVE-2026-56391					coreutils: GNU coreutils uniq: Denial of Service and information disclosure via out-of-bounds... https://avd.aquasec.com/nvd/cve-2026-56391
	CVE-2026-56392					coreutils: GNU coreutils unexpand: Denial of Service via crafted tab stop values... https://avd.aquasec.com/nvd/cve-2026-56392
dash	CVE-2026-102473			0.5.12-12		dash: dash: Super-polynomial backtracking in pmatch when libc fnmatch is disabled https://avd.aquasec.com/nvd/cve-2026-102473
	CVE-2026-102474					dash: dash: Heap out-of-bounds write in conv_escape via undersized Unicode escape reservation... https://avd.aquasec.com/nvd/cve-2026-102474
diffutils	CVE-2026-53910			1:3.10-4		diffutils: heap-based buffer overflow due to multiple signed integer overflows in line-mapping... https://avd.aquasec.com/nvd/cve-2026-53910
gcc-14-base	CVE-2026-102010	MEDIUM		14.2.0-19		gcc-toolset-15-gcc: gcc: gcc-toolset-16: gcc: Denial of Service via use-after-free in binary heap... https://avd.aquasec.com/nvd/cve-2026-102010
	CVE-2026-95619					gcc: libstdc++ integer overflow in `new` operator https://avd.aquasec.com/nvd/cve-2026-95619
libacl1	CVE-2026-54369	HIGH		2.3.2-2+b1		acl: Symlink traversal privilege escalation via libacl functions https://avd.aquasec.com/nvd/cve-2026-54369
	CVE-2026-54370	MEDIUM				acl: TOCTOU Symlink Traversal via getfacl/setfacl https://avd.aquasec.com/nvd/cve-2026-54370
libapt-pkg7.0	CVE-2011-3374	LOW		3.0.3		It was found that apt-key in apt, all versions, do not correctly... https://avd.aquasec.com/nvd/cve-2011-3374
libattr1	CVE-2026-54371	MEDIUM		1:2.5.2-3		attr: attr: Symlink Traversal Privilege Escalation via getfattr and setfattr https://avd.aquasec.com/nvd/cve-2026-54371
libblkid1	CVE-2026-76642	HIGH		2.41.5-0+deb13u1		util-linux: util-linux: failed external mount helper still runs privileged X-mount post-hooks https://avd.aquasec.com/nvd/cve-2026-76642
	CVE-2026-78408					util-linux: util-linux: nsenter --join-cgroup leaks root cgroup migration authority https://avd.aquasec.com/nvd/cve-2026-78408
	CVE-2026-78409					util-linux: util-linux: X-mount.subdir detached-tree resolution can escape via intermediate symlinks https://avd.aquasec.com/nvd/cve-2026-78409
	CVE-2026-78410					util-linux: util-linux: restricted bind mounts do not pin the source, allowing X-mount.owner/group/mode... https://avd.aquasec.com/nvd/cve-2026-78410
	CVE-2026-3184	MEDIUM				util-linux: util-linux: Access control bypass due to improper hostname canonicalization https://avd.aquasec.com/nvd/cve-2026-3184
	CVE-2022-0563	LOW				util-linux: partial disclosure of arbitrary files in chfn and chsh when compiled...

Figure 7. Trivy baseline detail — HIGH findings in util-linux, coreutils, acl and related base packages with no fixed version.

STEP B2 Secure Dockerfile

The Dockerfile was rewritten as a multi-stage build. Dependencies are installed in a throwaway build stage; the runtime stage is Chainguard's minimal Python image, which contains no shell, no package manager and no pip. Both base images are **pinned by digest** so a build cannot silently change. Application files are owned by root and read-only to the non-root runtime user, Flask's development server is replaced by **gunicorn**, a `.dockerignore` allow-lists only the two files the build needs, and a HEALTHCHECK is defined.

can-ehr-container/Dockerfile

```
# Hardened multi-stage build for the CAN EHR simulator.
# Stage 1 installs dependencies; stage 2 is a minimal Chainguard (Wolfi) image:
# no shell, no package manager, no pip, and 0 known CVEs at build time.

# ---- Build stage ----
FROM cgr.dev/chainguard/python:latest-dev@sha256:8c06d75b497c156bb7a42fedb6480fe2c1865e93538215e4a
    0f7bf99a03f1f99 AS build

ENV PIP_NO_CACHE_DIR=1 \
    PIP_DISABLE_PIP_VERSION_CHECK=1

# Root is only used in this throwaway build stage; the runtime stage runs as non-root
USER root
WORKDIR /build
COPY requirements.txt .
RUN pip install --no-compile --target /deps -r requirements.txt

# ---- Runtime stage ----
FROM cgr.dev/chainguard/python:latest@sha256:b7af1ae90e2fcfb5c32be03908e74d32fdfd64156c2b7c535bd3e
    497e7846d84

LABEL org.opencontainers.image.title="can-ehr-simulator" \
      org.opencontainers.image.description="Care Access Navigator EHR simulator (hardened)" \
      org.opencontainers.image.version="2.0"

ENV PYTHONPATH=/deps \
    PYTHONDONTWRITEBYTECODE=1 \
    PYTHONUNBUFFERED=1

WORKDIR /app
COPY --from=build --chown=root:root --chmod=0555 /deps /deps
COPY --chown=root:root --chmod=0444 app.py .

# UID 65532 = Chainguard "nonroot"; files above are root-owned and read-only to it
USER 65532:65532

EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD ["python3", "-c", "import urllib.request;
    urllib.request.urlopen('http://127.0.0.1:8080/health', timeout=2)"]

# Production WSGI server instead of the Flask development server
ENTRYPOINT ["python3", "-m", "gunicorn"]
CMD ["--bind", "0.0.0.0:8080", "--workers", "2", "--worker-tmp-dir", "/tmp", "--access-logfile",
    "-", "app:app"]
```

STEP B3

Docker security settings

The container is run from `docker-compose.secure.yml` with a read-only root filesystem (only a small noexec `/tmp`), every Linux capability dropped, privilege escalation blocked, process/memory/CPU limits, the port published on localhost only, an isolated bridge network and log rotation.

can-ehr-container/docker-compose.secure.yml

```
# Hardened runtime settings for the CAN EHR simulator.
services:
  ehr-simulator:
    image: can-ehr-simulator:2.0
    build: .
    container_name: can-ehr-simulator
    user: "65532:65532"
    read_only: true                # immutable root filesystem
    tmpfs:
      - /tmp:rw,noexec,nosuid,nodev,size=16m
    cap_drop: [ALL]                # no Linux capabilities needed (port 8080 > 1024)
    security_opt:
      - no-new-privileges:true     # block setuid/privilege escalation
    pids_limit: 100                # fork-bomb protection
    mem_limit: 256m
    cpus: 0.5
    ports:
      - "127.0.0.1:8081:8080"      # bind to localhost only, not all interfaces
    networks: [ehr-net]
    restart: on-failure:3
    logging:
      driver: json-file
      options: { max-size: "10m", max-file: "3" }

networks:
  ehr-net:
    driver: bridge
```

STEP B4 Runtime verification of the Docker settings

reports/hardened/runtime-security-tests.txt (tests run inside the live container)

```
User=65532:65532 ReadOnly=true CapDrop=[ALL] SecOpt=[no-new-privileges:true] Pids=100
    Mem=268435456 NanoCPUs=500000000 Privileged=false Ports=map[8080/tcp:[map[HostIp:127.0.0.1
    HostPort:8081]]]
shell available?  executable file not found in $PATH
pip available?    /usr/bin/python3: No module named pip
uid/gid:          65532 65532
write /app:       OSError: [Errno 30] Read-only file system: '/app/x'
modify app.py:    PermissionError: [Errno 13] Permission denied: '/app/app.py'
write /tmp:       allowed (tmpfs, noexec,nosuid,nodev)
caps / nnp:       CapEff:   0000000000000000 NoNewPrivs:    1
fork bomb limit:  blocked after 96 processes: Resource temporarily unavailable
still healthy?    {"service":"ehr-simulator-healthcheck","status":"ok"}

reachable from LAN IP? no (bound to 127.0.0.1 only)
```

STEP B5 Hardened scan — after

The rebuilt image scans clean: **0 vulnerabilities** in the OS layer and in every Python package, 0 secrets and 0 Dockerfile misconfigurations. The image also shrank from 223 MB to 105 MB, and a CycloneDX SBOM was generated (`reports/hardened/sbom.cdx.json`).

```
Adult@DAEDMAC1009 dae_6mon_coh2_AC % cat ~/Desktop/dae_6mon_coh2_AC/can-ehr-container/reports/hardened/trivy-ima

Report Summary

┌──────────────────────────────────────────────┬────────────┬─────────────────┬─────────┐
│                    Target                    │    Type    │ Vulnerabilities │ Secrets │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ can-ehr-simulator:2.0 (wolfi 20230201)       │   wolfi    │        0        │    -    │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ deps/blinker-1.9.0.dist-info/METADATA        │ python-pkg │        0        │    -    │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ deps/click-8.5.0.dist-info/METADATA          │ python-pkg │        0        │    -    │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ deps/flask-3.1.3.dist-info/METADATA          │ python-pkg │        0        │    -    │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ deps/gunicorn-23.0.0.dist-info/METADATA      │ python-pkg │        0        │    -    │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ deps/itsdangerous-2.2.0.dist-info/METADATA   │ python-pkg │        0        │    -    │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ deps/jinja2-3.1.6.dist-info/METADATA         │ python-pkg │        0        │    -    │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ deps/markupsafe-3.0.4.dist-info/METADATA     │ python-pkg │        0        │    -    │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ deps/packaging-26.3.dist-info/METADATA       │ python-pkg │        0        │    -    │
├──────────────────────────────────────────────┼────────────┼─────────────────┼─────────┤
│ deps/werkzeug-3.1.9.dist-info/METADATA       │ python-pkg │        0        │    -    │
└──────────────────────────────────────────────┴────────────┴─────────────────┴─────────┘
Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

Adult@DAEDMAC1009 dae_6mon_coh2_AC %
```

Figure 8. Trivy scan of the hardened image can-ehr-simulator:2.0 (wolfi) — 0 vulnerabilities in every target.

Metric	Baseline (python:3.12-slim)	Hardened (Chainguard, multi-stage)
Total vulnerabilities	172	**0**
HIGH / CRITICAL	44 / 0	0 / 0
Image size	223 MB	105 MB
Shell / pip / package manager	Yes / yes / apt	None
Web server	Flask development server	gunicorn 23.0.0
Runtime user	appuser (non-root)	UID 65532 (non-root), read-only files

Issue encountered: Google's distroless image was tried first, but Trivy showed it was *worse* (283 findings, including 2 CRITICAL) because it is built on older Debian 12 packages. Scanning every candidate base image before adopting it caught this; Chainguard's Wolfi-based image was chosen instead.

Part C — Serverless Function: IAM & Network Isolation

A Python HTTP-triggered Azure Function (`ehr_status`) reads synthetic referral records from blob storage. The function, its identity, its storage and its network were all locked down so that it holds no secrets, has only the access it needs, and is reachable only from approved locations.

STEP C1 Function App on Flex Consumption

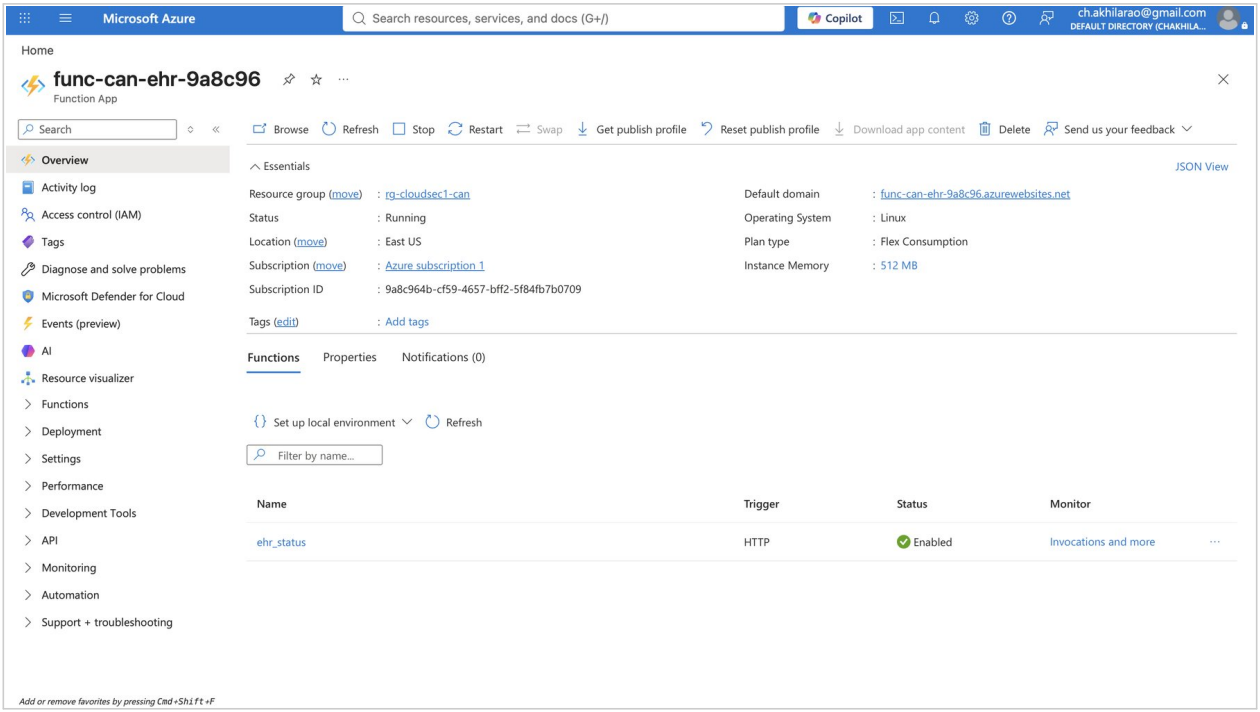


Figure 9. Function App func-can-ehr-9a8c96 — Flex Consumption plan, Linux, 512 MB instances, ehr_status HTTP function enabled.

STEP C2 IAM — managed identity with a single least-privilege role

The function authenticates to Azure with a dedicated **user-assigned managed identity**, **id-can-func**. Azure issues and rotates its tokens, so there are no passwords, keys or connection strings in code or configuration. The identity holds exactly **one role** — *Storage Blob Data Owner* — scoped to **one storage account** (stcanfunc9a8c964b), and nothing at subscription or resource-group level. The Functions host itself also uses this identity (`AzureWebJobsStorage__credential = managedidentity`).

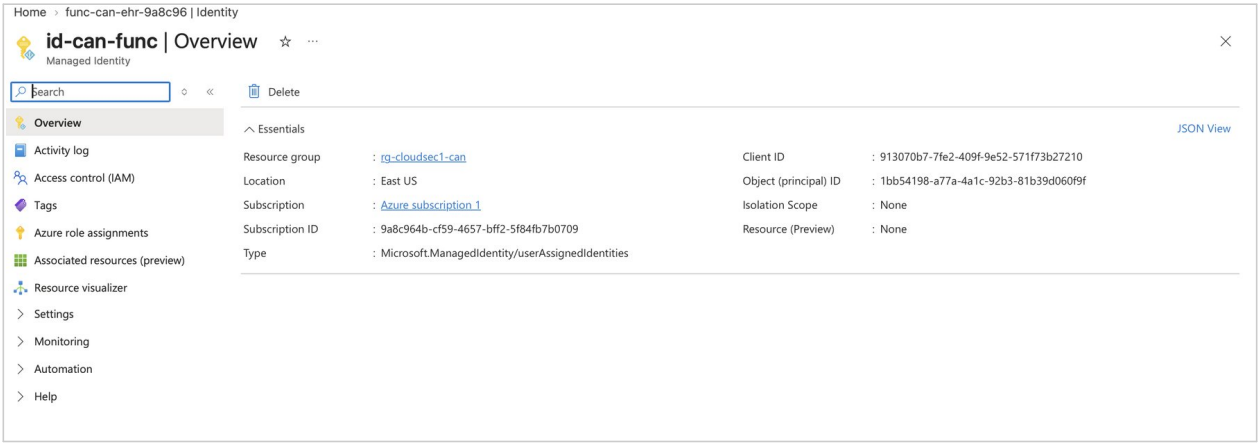


Figure 10. User-assigned managed identity id-can-func (client ID 913070b7-…).

az role assignment list — id-can-func holds exactly one role, on one storage account

```
== Every Azure role held by id-can-func (all scopes, incl. inherited) ==
Storage Blob Data Owner    storageAccounts/stcanfunc9a8c964b

== Role assignments on storage account stcanfunc9a8c964b ==
id-can-func  (ServicePrincipal 913070b7-7fe2-...)   Storage Blob Data Owner
admin user   (ch.akhilarao, User)                   Storage Blob Data Contributor  [for uploading
    test data]
```

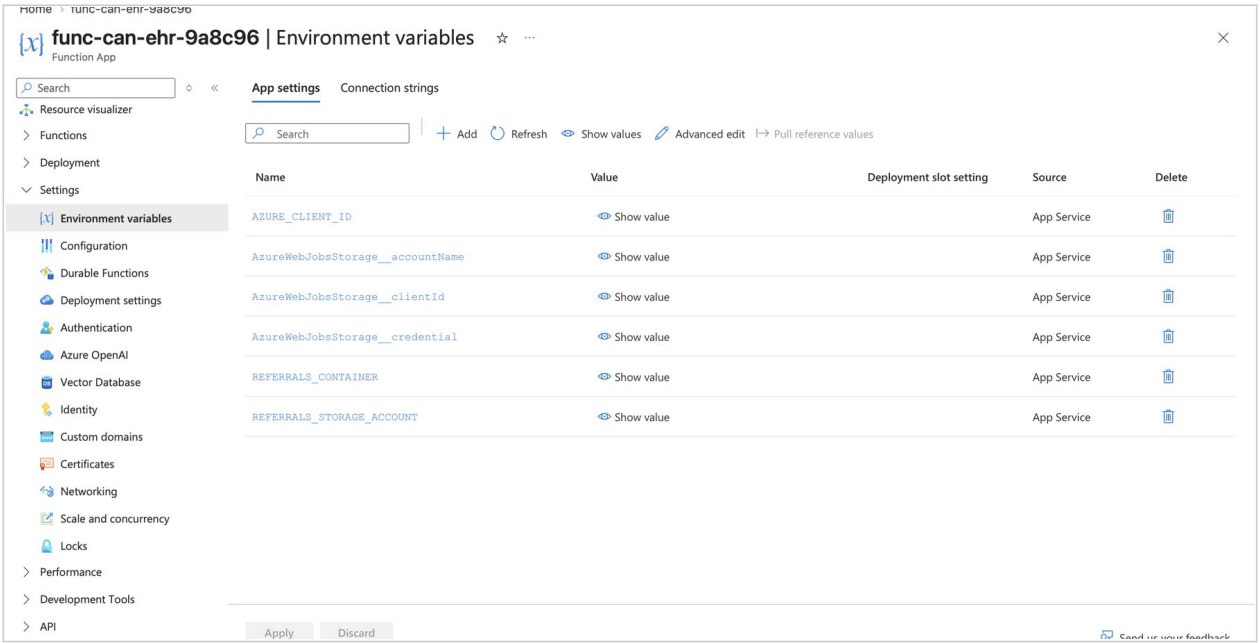


Figure 11. Function app settings — identity-based storage settings only; no AccountKey, connection string or secret is stored.

> **Issue encountered:** Azure's function-creation wizard inserted a storage **account key** into AzureWebJobsStorage by default. It was replaced with the three identity-based settings shown above and deleted, and key-based access was then disabled on the storage account, so the original key can no longer be used even if leaked.

STEP C3 Network isolation

Outbound, the function is integrated into the private subnet **snet-func** (with NSG func-sg), so its calls to storage leave through the VNet. Inbound, access restrictions allow only the administrator's IP and deny everything else, for both the function endpoint and its deployment (SCM) endpoint. The storage account's firewall denies by default and accepts only snet-func and the administrator IP, with no trusted-service bypass.

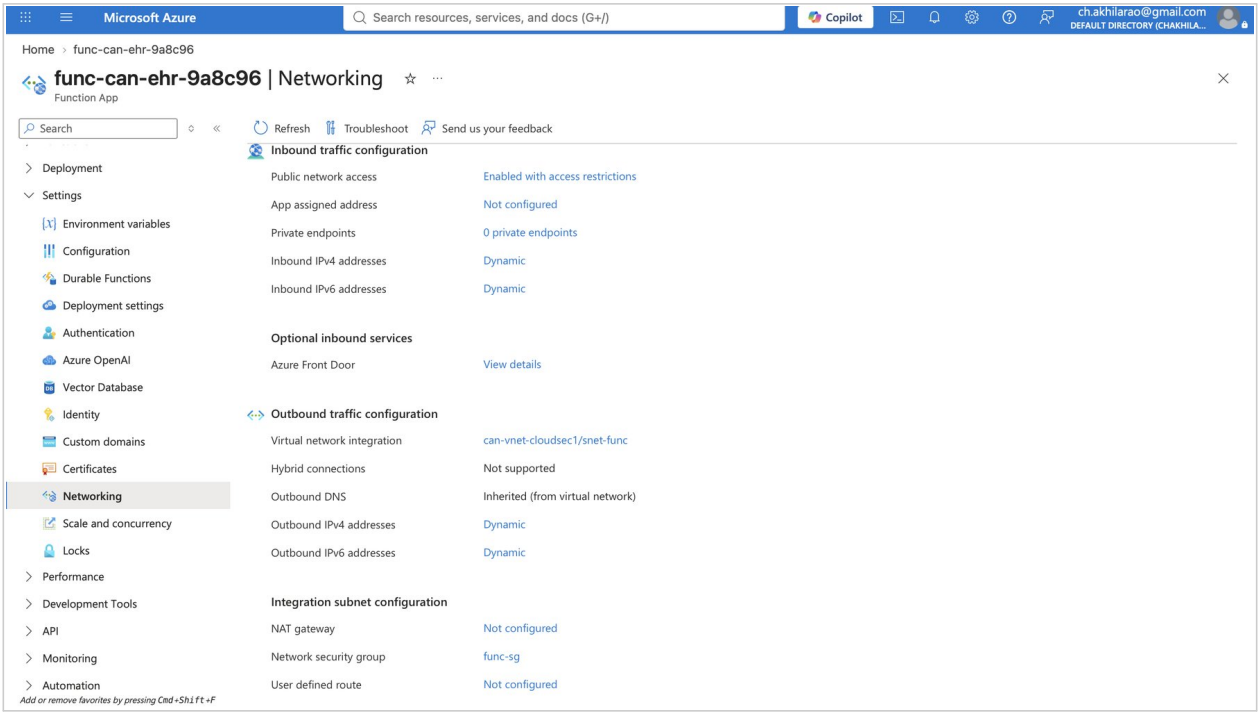


Figure 12. Function networking — public access 'Enabled with access restrictions'; outbound VNet integration to can-vnet-cloudsec1/snet-func with NSG func-sg.

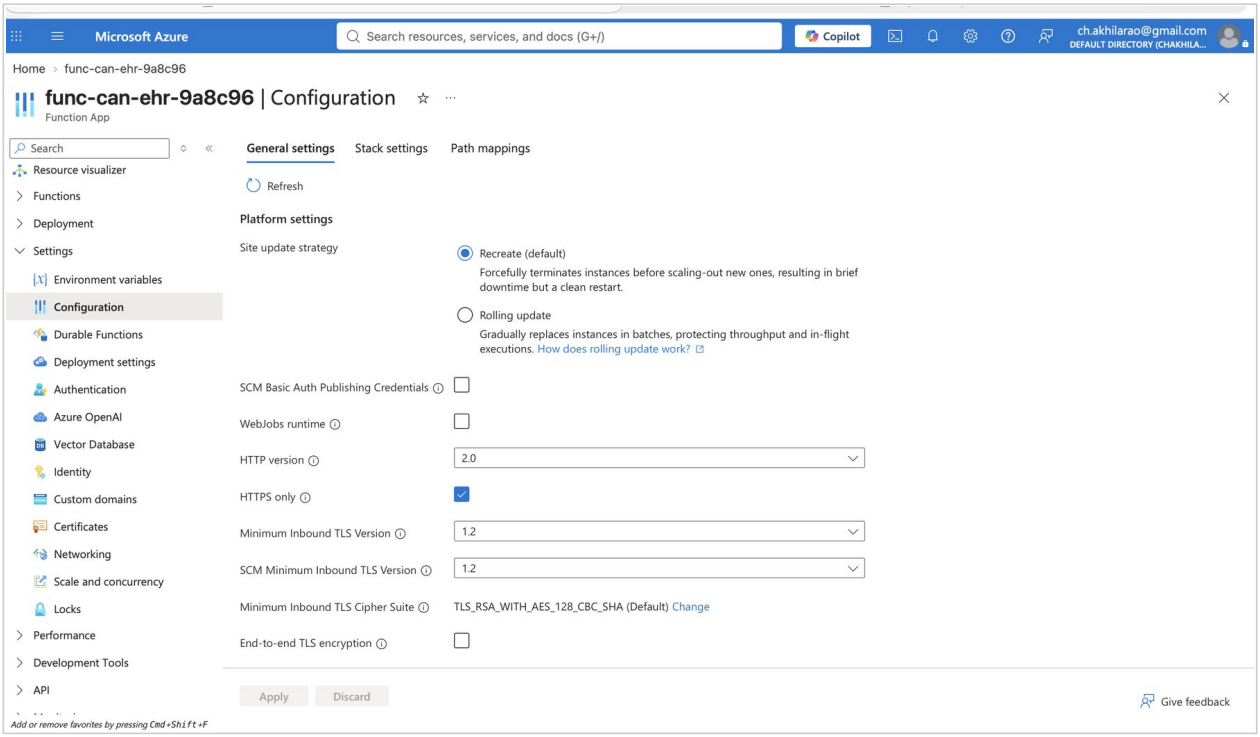


Figure 13. Function configuration — HTTPS only, minimum inbound TLS 1.2 (site and SCM), SCM basic-auth publishing credentials disabled.

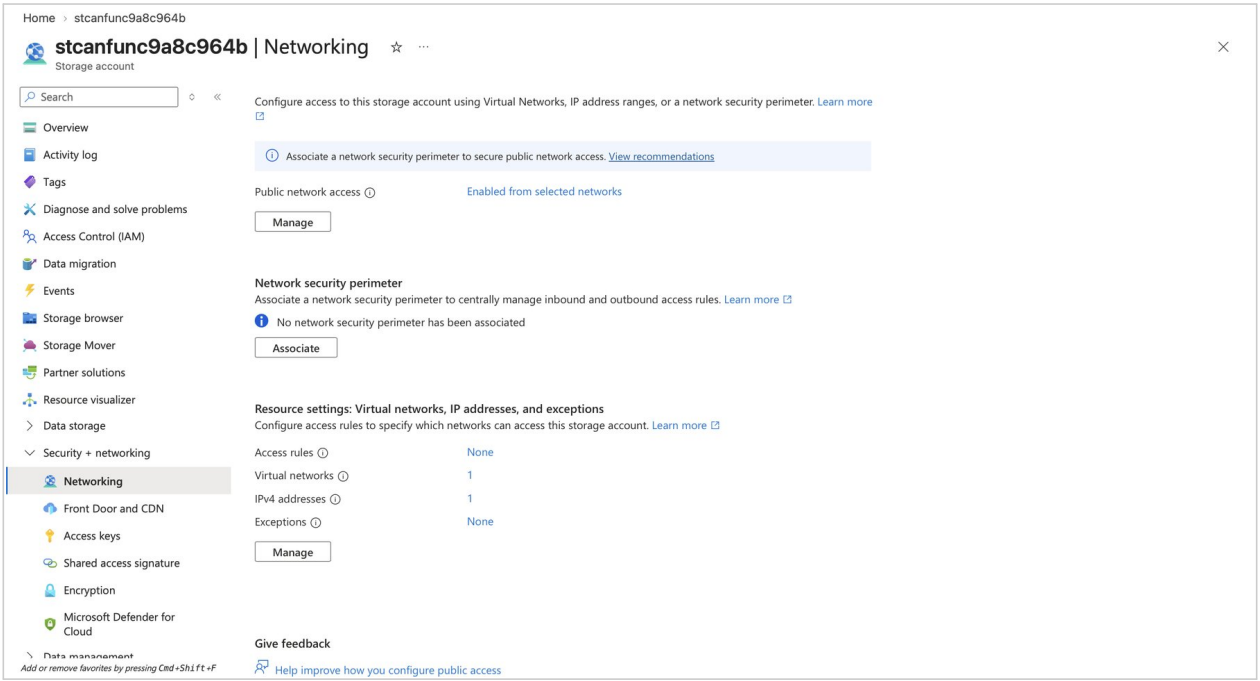


Figure 14. Storage networking — public network access 'Enabled from selected networks': 1 virtual network rule (snet-func) and 1 IPv4 rule (admin IP); no exceptions.

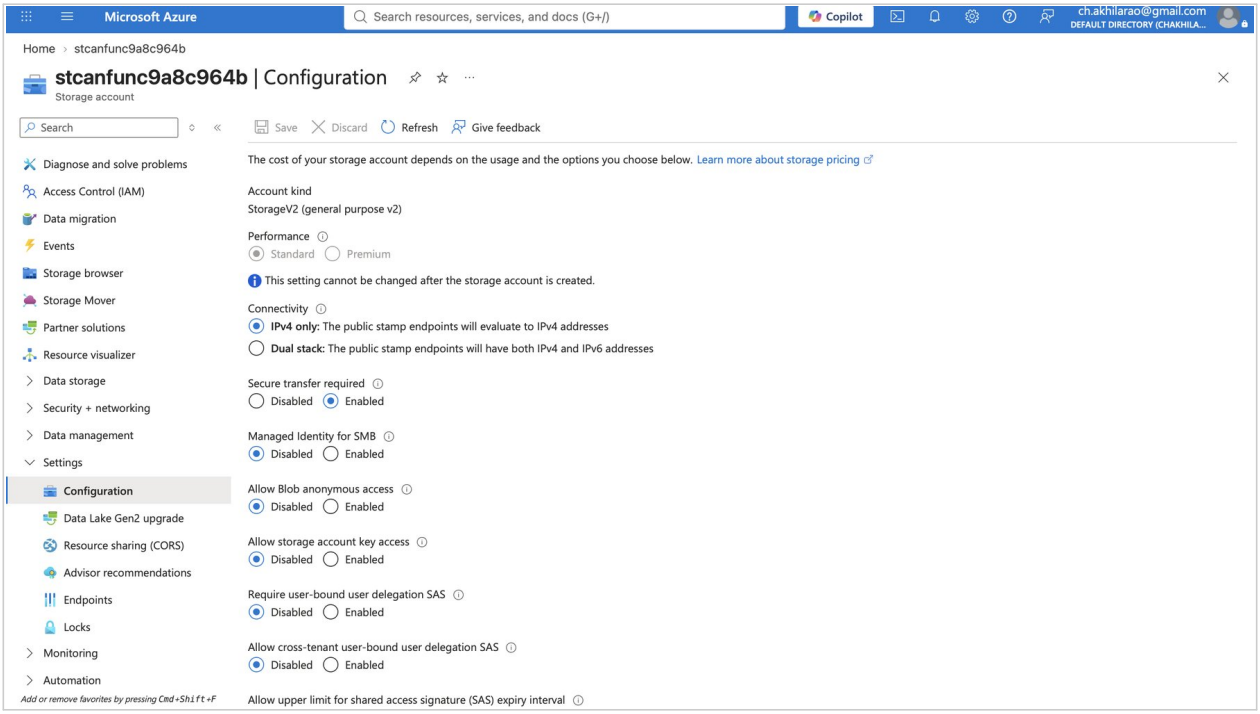


Figure 15. Storage configuration — secure transfer required, anonymous blob access disabled, storage account key access disabled.

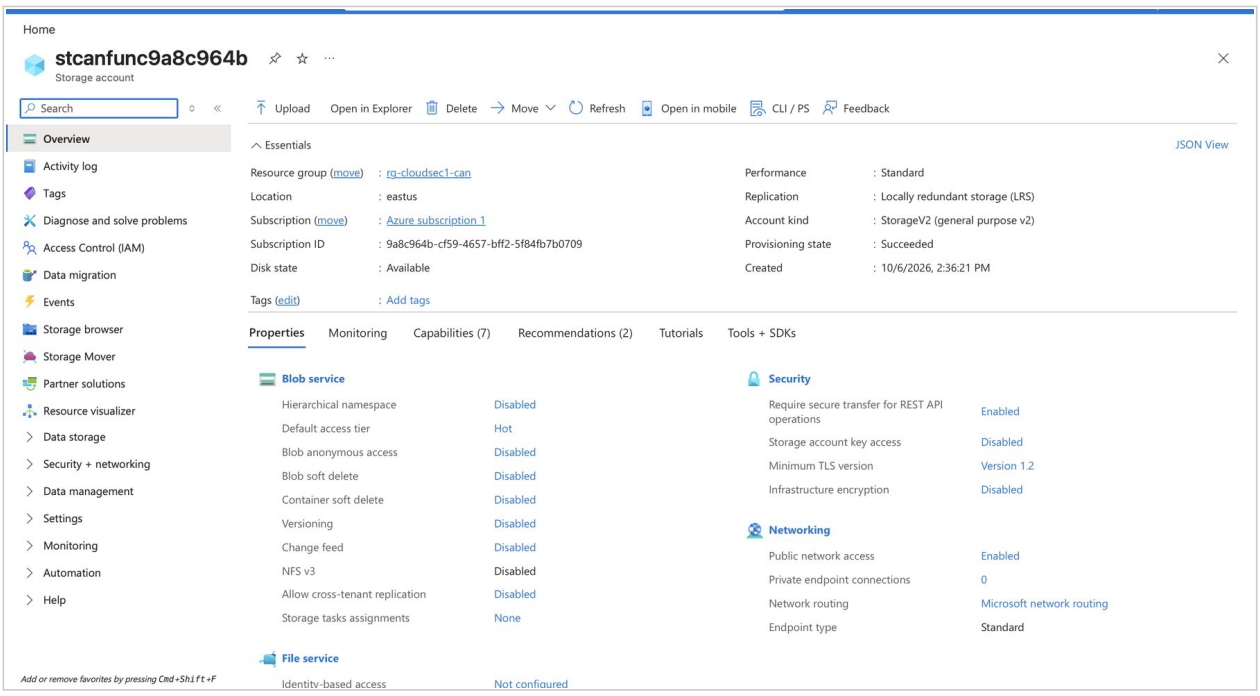


Figure 16. Storage overview — key access Disabled, minimum TLS 1.2, anonymous access Disabled.

STEP C4 Security tests

can-ehr-serverless/evidence/security-tests.txt

```
== Function inbound (from allowed IP 24.189.77.111) ==
no key:            401
with key:          {"service": "CAN EHR status", "status": "ok", "auth": "managed-identity",
    "referrals_container": "referrals", "referral_count": 3}  [200]
plain HTTP:        301 -> https://func-can-ehr-9a8c96.azurewebsites.net/api/ehr-status
TLS 1.1 forced:    handshake refused (min TLS 1.2)
== Storage firewall ==
anonymous read:    409 (public blob access disabled)
shared key access: not permitted
== Negative test: only 203.0.113.10 allowed; calling from 24.189.77.111 ==
with valid key:    <h1 id="unavailable">Error 403 - Forbidden</h1> ...  [403]
```

For the negative test, the allow rule was temporarily changed to a different IP (203.0.113.10). A call from the administrator's real IP with a **valid function key** was then refused with 403 — proving the network restriction is enforced independently of the key. The rule was restored afterwards.

Issues Encountered & Resolutions

Issue	Resolution
Bastion host (Basic) and Standard container registry were costing ~$5/day against limited credit	Bastion and its public IP deleted; ACR downgraded Standard → Basic. VM SSH instead restricted by NSG + ufw to the admin IP.
Standard_B1s (and other B-series) unavailable in East US due to capacity/quota limits on the subscription	Queried unrestricted SKUs per region; deployed Standard_F1als_v7 in North Central US on its own VNet.
Distroless base image scanned worse than the original (283 findings, 2 CRITICAL)	Rejected after scanning; switched to Chainguard Wolfi image (0 findings).
Chainguard build stage runs as non-root, so pip could not write /deps	USER root in the throwaway build stage only; runtime stage remains UID 65532.
Function created with a storage account key in its settings	Replaced with managed-identity settings; key access disabled on the storage account.
Function creation silently granted the identity a second role (Storage Blob Data Contributor) alongside Owner	Found by auditing role assignments; redundant role removed and the function re-tested (200 OK), leaving exactly one role.
An IPv6 'other IP' test was invalid (host has no IPv6, so it still used the allowed IPv4)	Discarded; replaced with a controlled test that temporarily allowed only a different IP (403).

Compliance & Control Mapping

Control area	HIPAA Security Rule	NIST CSF 2.0	CAN evidence
Automated patching	§164.308(a)(5)(ii)(B) Protection from malicious software	PR.PS-02 (software maintained)	Steps A1–A2, Fig. 2
Host & network firewall	§164.312(e)(1) Transmission security	PR.IR-01 (networks protected)	Step A3, Fig. 3, IP flow verify
Secure SSH / authentication	§164.312(d) Person or entity authentication; §164.312(a)(1) Access control	PR.AA-03 (users authenticated)	Steps A4–A5, Fig. 4, 5
Logging & brute-force detection	§164.312(b) Audit controls	DE.CM-09 / PR.PS-04 (logs generated)	fail2ban, ufw logging, sshd VERBOSE
Container vulnerability scanning	§164.308(a)(1)(ii)(A) Risk analysis	ID.RA-01 (vulnerabilities identified)	Steps B1, B5, Fig. 6–8
Secure container configuration	§164.308(a)(1)(ii)(B) Risk management	PR.PS-01 (configuration management)	Steps B2–B4
Least-privilege workload identity	§164.308(a)(4) Information access management	PR.AA-05 (access permissions managed)	Step C2, Fig. 10, 11
Serverless network isolation	§164.312(e)(1) Transmission security	PR.IR-01 (networks protected)	Steps C3–C4, Fig. 12–16

Implementation Status

Implemented & verified: VM hardening (two-layer automated patching, NSG + ufw firewall, key-only SSH with fail2ban, Trusted Launch); container hardening (multi-stage digest-pinned Chainguard image, hardened runtime settings, Trivy 172 → 0); serverless isolation (managed identity with a single scoped role, no keys, VNet integration, storage firewall, IP restrictions, HTTPS/TLS 1.2).

Known gaps & next steps

- The container registry's admin user is still enabled because the existing container instance (aci-app-test) pulls with it. Next step: recreate the instance with a managed identity holding AcrPull, then disable the admin user.
- Storage and function are reached over public endpoints restricted by firewall/IP rules. Private endpoints would remove public exposure entirely but add ~$7/month each; deferred under the cost constraint.
- The VM uses a public IP restricted to one admin address. In production, access would be through Azure Bastion or a VPN with no public IP (Bastion was removed here for cost).
- Trivy should run automatically in CI on every build so a new vulnerability fails the build.